

# AI and Streamlit

From LLM to Interactive Web  
Applications

Introduction to Streamlit

Introduction to LM Studio

Use Case

---

# Why ?

Historically, data analysts and scientists haven't had the skills to build web applications

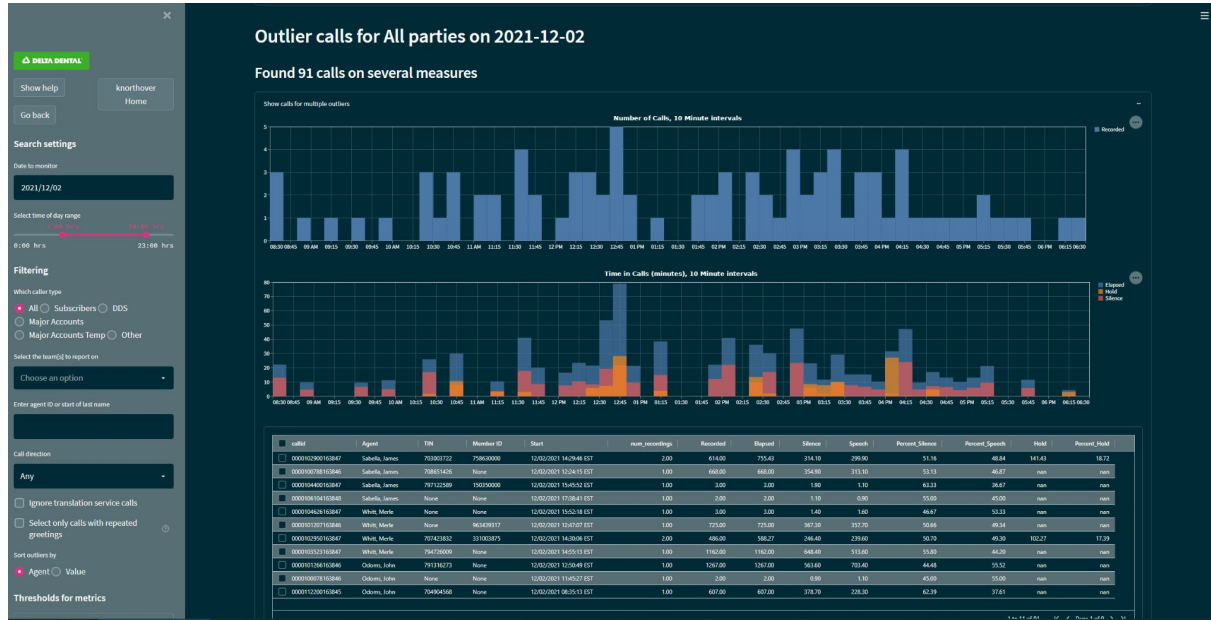
However, there's no better tool for making demos and sharing your work with business teams

This eliminates the need for software engineers to build in-house tools.

And here comes Streamlit !

# What is Streamlit ?

Open-source Python library that makes it easy to create and share custom web apps for machine learning and data science projects.



# Key Features

**Simplicity:** No front-end experience required; write apps entirely in Python.

**Rapid Development:** Real-time updates with hot-reloading.

**Widgets:** Interactive widgets like sliders, buttons, and text inputs.

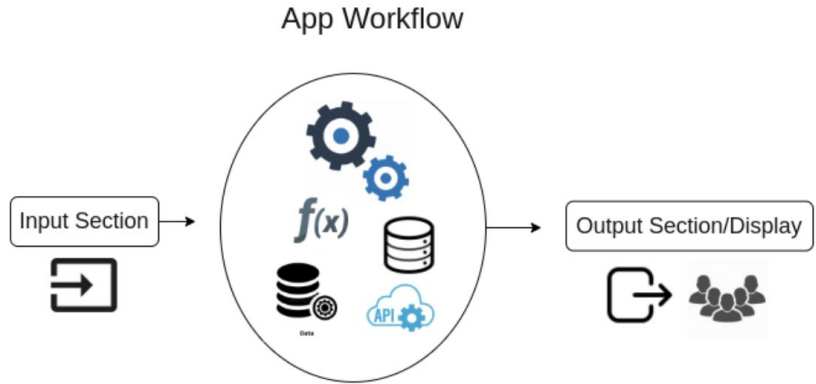
**Visual:** Use icons or images representing simplicity, speed, and interactivity.

# Why Use Streamlit?

Quickly transform data scripts into interactive apps.

Enhance data storytelling with interactive visualizations.

Facilitate collaboration by sharing apps easily.

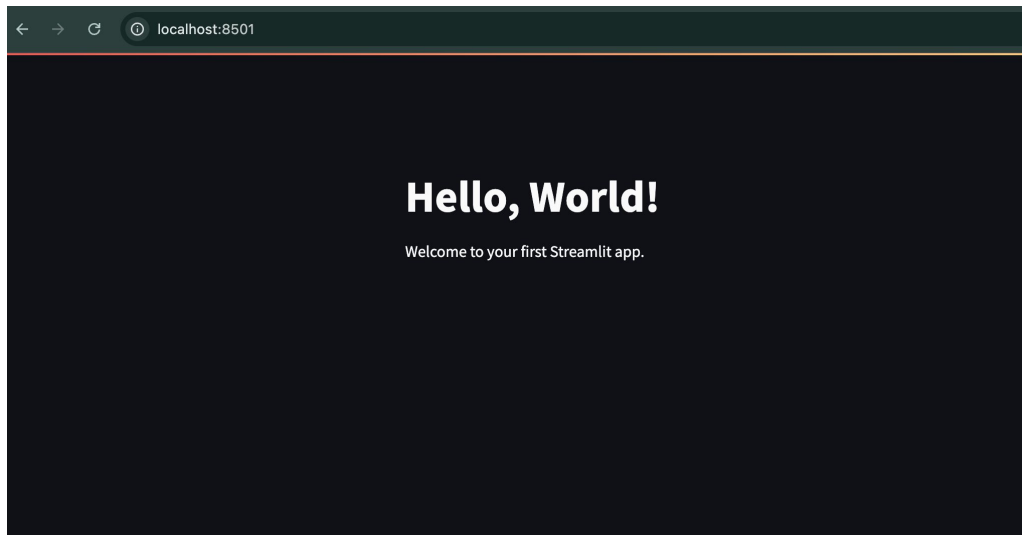


# Basic Syntax and Structure

Open-source Python library that makes it easy to create and share custom web apps for machine learning and data science projects.

```
import streamlit as st

st.title('Hello, World!')
st.write('Welcome to your first Streamlit app.')
```

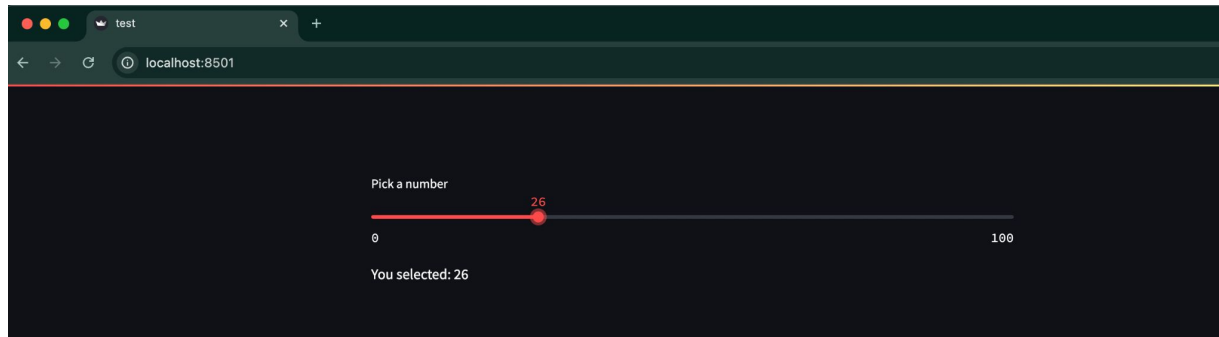


# Interactive Widgets

Sliders, Buttons, Text Input etc....

```
import streamlit as st

number = st.slider('Pick a number', 0, 100)
st.write(f'You selected: {number}')
```



# Ressources to get started

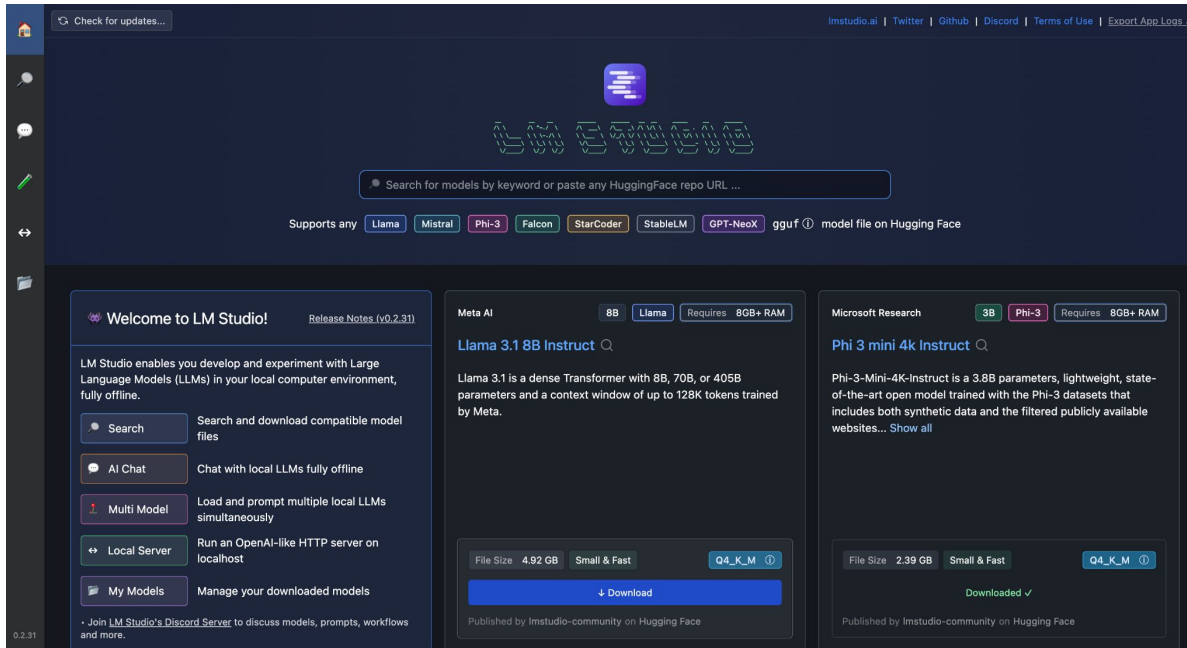
Official Documentation: <https://docs.streamlit.io>

Community Forums: <https://discuss.streamlit.io>



# What is LM Studio ?

Open-source application that allows users to run Large Language Models on their local machines, entirely offline



# Key Features

**Offline Operation:** Run LLMs without an internet connection, ensuring data privacy.

**Model Support:** Supports various architectures, including Llama, Mistral, Phi, Gemma, and Qwen.

**Interactive Chat UI:** Engage with models through an intuitive chat interface.

**Document Interaction:** Chat with your local documents

# Why LM Studio ?

**Privacy:** Data remains local, ensuring confidentiality.

**Accessibility:** No need for high-end cloud services; run models on personal hardware.

**Cost-Effective:** Eliminates the need for expensive cloud-based solutions.

# Getting Started with LM Studio

**Download and Install:** Obtain the latest version from the official website.

**Download a Model:** Use the Discover tab to find and download compatible models.

# Embeddings

Embeddings are numerical vector representations of text, images, or other data types.

They transform complex data into high-dimensional vectors that capture semantic meaning (not just keywords).

Similar meanings → vectors close together in the vector space.

Used by AI systems to measure semantic similarity, perform clustering, retrieval, and more.

# RAG with ChromaDB

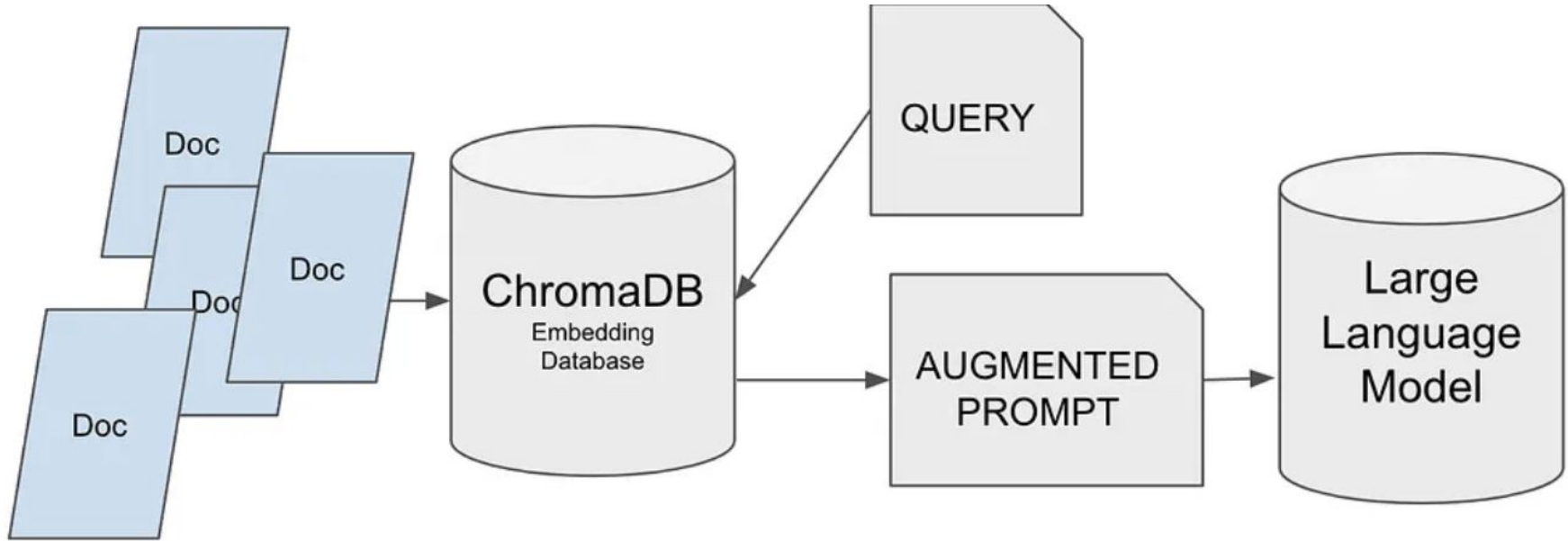
## Lightweight Vector Database

ChromaDB is an open-source vector database for storing and searching embeddings locally or in small deployments.

It is designed to be simple to install and use (e.g., `pip install chromadb`).

You can embed text and store pairs of (text chunk + embedding vector + metadata).

# RAG with ChromaDB - Embeddings



# RAG with ChromaDB - Embeddings

**Chunking Documents** - Split large texts (e.g., book chapters) into small chunks (e.g., 300–500 words)

**Generating Embeddings** - Use an embedding model to convert chunks into vectors — numeric semantic representations

**Store in ChromaDB** - Add documents + embeddings + optional metadata into a ChromaDB collection

## Query-Time Retrieval :

- Embed the user query into a vector

- Search the vector database for nearest neighbors (most semantically similar chunks)

- Retrieve those chunks to provide context for a summarization or answer generation step (LLM)



# Assignment

## Build a Chatbot based on a LLM in a Streamlit Web App

- HomePage
- Chat Page
- Model Choice

## Chat with Documents :

- Download Othello (<https://www.gutenberg.org/>)
- Chunk the book, generate embedding and store them in Chroma DB
- Your chatbot must be able to answer questions about Othello

# Assignment

## Two Options :

- You are using OpenAI API and your code can work with a .env file and you will explain the name of the environment variable in the readme
- You are using a model downloaded on hugging face : use the transformers library

This applies to both your embedding and your chatbot models

# Useful links

## Streamlit

- Official documentation <https://docs.streamlit.io>
- Community forums <https://discuss.streamlit.io>

## LM Studio

- You can use mistralai/ministral-3-3b
- LM Studio App documentation <https://lmstudio.ai/docs/app>
- Python SDK documentation <https://lmstudio.ai/docs/python>

## ChromaDB

- <https://docs.trychroma.com/docs/overview/getting-started>

# Assignment

## Deliverables :

- The python code to generate your vector database and the streamlit app
- A requirements.txt file with all the libraries used to run your code
- A readme explaining how to use your application
- The code has comments and docstrings

# Rating Grid

- Respect deadline : 2pts
- The chat is working and cites its sources : 6pts
- The chat gives relevant answer about Othello : 2pts
- The chat can handle history : 3pts
- Your functions are commented and have docstrings : 2pts
- No functions have more than 25 lines : 2pts
- Your streamlit app has an homepage presenting the chat : 2pts
- Your app is using at least 3 widgets: 1pt

If you're using Chat GPT to write your code... you'd better make sure it's fully commented by you